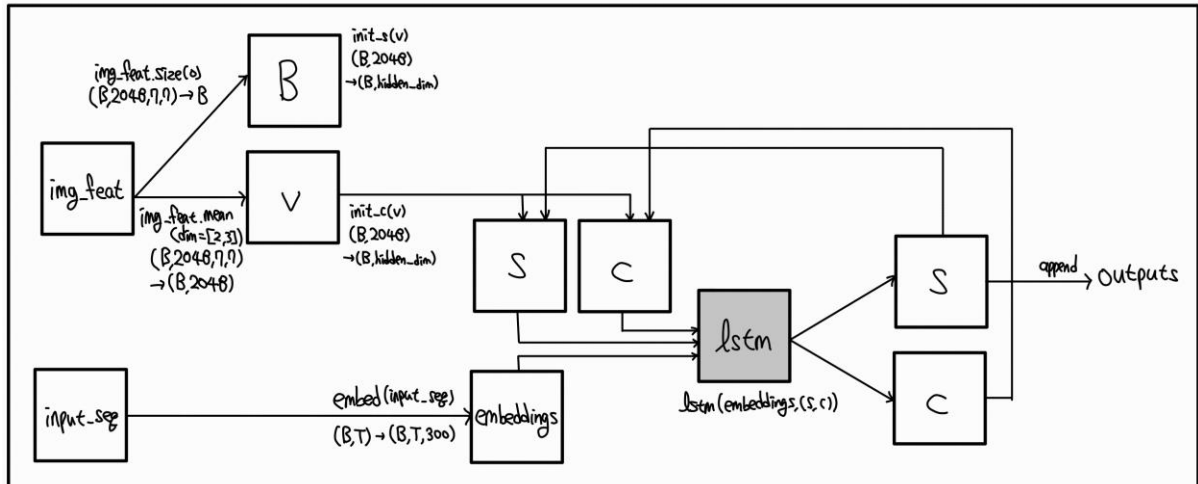
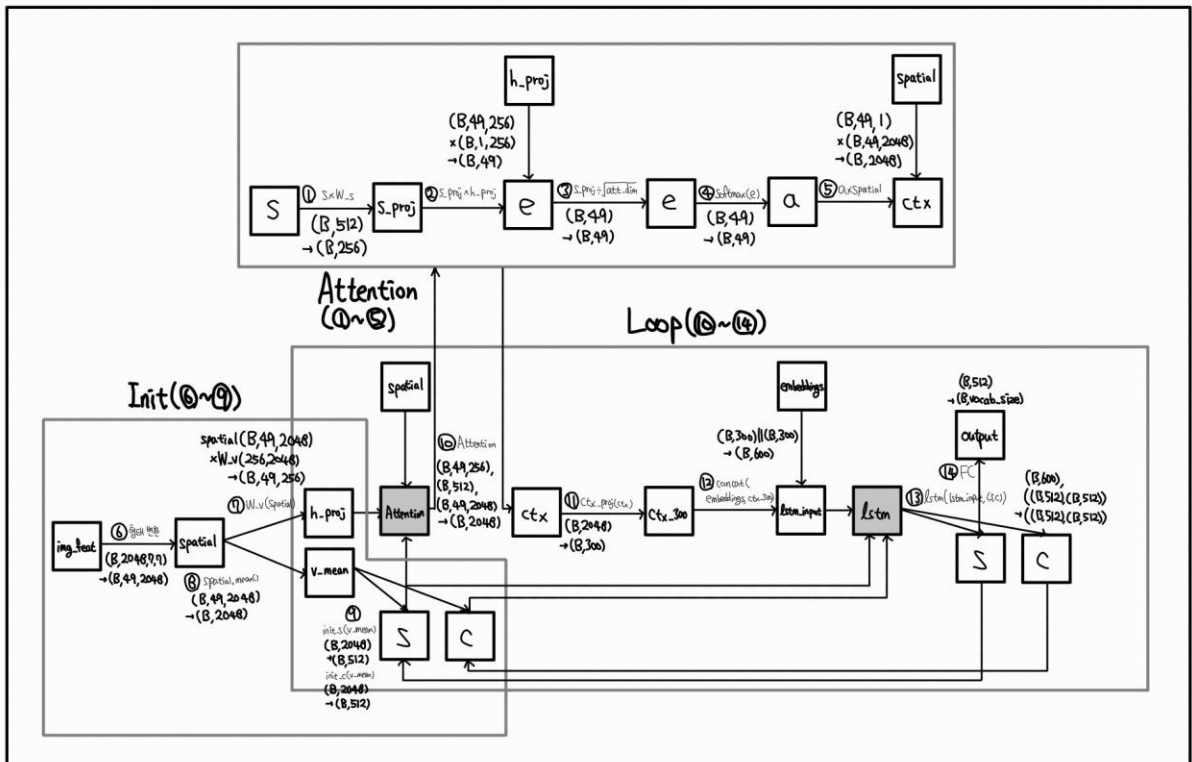


Part 1. CaptionBase, CaptionAttention의 아키텍처 다이어그램

CaptionBase



CaptionAttention



Part 2. 코드 분석

① `s_proj = self.W_s(s)`

역할: 이전 시점의 decoder hidden state s_{t-1} 를 attention 차원(att_dim)으로 투영한다. 이 벡터는 각 spatial feature와 비교하기 위한 query 역할을 한다.

Shape: (B, 512) -> (B, 256)

② `e = (h_proj * s_proj.unsqueeze(1)).sum(dim=2)`

역할: 각 spatial location마다 query와의 내적으로 유사도를 계산해 attention score를 만든다. 즉, 49개 위치 각각이 현재 문맥과 얼마나 관련 있는지 측정하는 단계다.

Shape: (B, 49, 256), (B, 1, 256) -> (B, 49)

③ `e = e / (self.att_dim ** 0.5)`

역할: attention score를 $\sqrt{\text{att_dim}}$ 으로 나누어 softmax가 너무 뽀족해지지 않도록 안정화한다. softmax는 지수함수를 이용하므로 값이 너무 크면 한 위치로 치우칠 수 있어, 이 스케일링은 gradient 불안정을 줄여준다.

Shape: (B, 49) -> (B, 49)

④ `a = F.softmax(e, dim=1)`

역할: score를 softmax로 정규화해 49개 spatial 위치에 대한 attention weight를 얻는다. 각 값은 0~1 사이의 확률이며 합은 1이 된다.

Shape: (B, 49) -> (B, 49)

⑤ `ctx = (a.unsqueeze(2) * spatial).sum(dim=1)`

역할: attention weight로 spatial feature를 가중합해 context vector ctx를 만든다. 중요한 위치는 더 크게 반영되고 덜 중요한 위치는 약하게 반영된다.

Shape: (B, 49, 1) * (B, 49, 2048) -> (B, 2048)

⑥ `spatial = img_feat.view(B, 2048, -1).permute(0, 2, 1).contiguous()`

역할: ResNet feature map을 7x7 공간 격자 기준으로 펼쳐서 (49, 2048) sequence로 바꾼다. 이렇게 해야 attention이 각 spatial location을 개별적으로 볼 수 있다.

Shape: (B, 2048, 7, 7) -> (B, 49, 2048)

⑦ `h_proj = self.W_v(spatial)`

역할: 각 spatial feature를 attention key용 차원(att_dim)으로 투영한다. query(s_proj)와 같은 차원으로 맞춰 내적을 계산할 수 있게 만든다.

Shape: (B, 49, 2048) -> (B, 49, 256)

⑧ `v_mean = spatial.mean(dim=1)`

역할: 모든 spatial location의 feature를 평균내어 이미지 전체의 전역 요약 정보를 생성한다. 이 값은 decoder 초기 state를 만드는 데 사용된다.

Shape: (B, 49, 2048) -> (B, 2048)

⑨ `s = self.init_s(v_mean)`

역할: 전역 이미지 요약 벡터를 이용해 decoder의 초기 hidden state s_0를 생성한다. 이 초기값은 caption generation을 시작할 때의 첫 문맥 역할을 한다.

Shape: (B, 2048) -> (B, 512)

⑩ `c = self.init_c(v_mean)`

역할: 전역 이미지 요약 벡터를 이용해 decoder의 초기 cell state c_0를 생성한다. 이 초기값은 hidden state와 함께 LSTM의 초기 문맥을 이룬다.

Shape: (B, 2048) -> (B, 512)

⑩ `ctx, _ = self._attention(h_proj, s, spatial)`

역할: 현재 decoder state `s`를 기준으로 attention을 다시 계산해 attention context `ctx`를 얻는다. 반복되는 decoder loop 안에서 매 time step마다 다른 spatial location에 주목할 수 있도록 호출된다.

Shape: `h_proj(B, 49, 256)`, `s(B, 512)`, `spatial(B, 49, 2048)` -> `(B, 2048)`

⑪ `ctx_300 = self.ctx_proj(ctx)`

역할: context vector를 단어 embedding 차원(300)으로 투영한다. 이후 현재 단어 embedding과 concat하기 위해 차원을 맞추는 단계다.

Shape: `(B, 2048)` -> `(B, 300)`

⑫ `lstm_input = torch.cat([embeddings[:, t], ctx_300], dim=1)`

역할: 현재 시점의 단어 embedding과 attention context를 이어 붙여 LSTM 입력을 만든다. 즉, 현재 시점 입력 단어의 embedding `embeddings[:, t]`와 이미지에서 주목한 정보 `ctx_300`을 함께 사용한다.

Shape: `(B, 300) + (B, 300)` -> `(B, 600)`

⑬ `s, c = self.lstm(lstm_input, (s, c))`

역할: LSTMCell이 이전 hidden/cell state와 현재 입력을 받아 새로운 decoder state를 갱신한다. 현재 입력과 이전 (hidden state, cell state)를 받아 새로운 (hidden state, cell state)를 만든다.

Shape: `(B, 600)`, `((B, 512), (B, 512))` -> `((B, 512), (B, 512))`

⑭ `outputs.append(self.fc_out(s))`

역할: 갱신된 hidden state `s`를 vocabulary logits으로 변환해 다음 단어 분포를 만든다. 학습 시에는 CrossEntropyLoss에 들어가고, 생성 시에는 argmax로 다음 단어를 고른다.

Shape: `(B, 512)` -> `(B, vocab_size)`